

DoS Stress-Testing & Auto-Mitigation for AI Inference Servers

Cost-aware admission control in front of a vLLM endpoint

Team Phishers

Mena Abdelmoneim · Salama Albedwawi · Rewan Ramadan

vLLM 0.29.0 / Qwen2.5-0.5B-Instruct / NVIDIA Tesla T4
github.com/mena-04/DoS-Stress-Testing

Under an eight-way flood of 512-token generations, turning the gateway on cut legitimate p95 latency from **9.55 s to 1.32 s** while raising the number of legitimate requests answered from **3 to 15** — and rejected none of them. Latency and throughput improving together is what separates protecting users from refusing them.

7.2×

lower legitimate p95 under attack

5.0×

more legitimate requests served

50%

of attacker traffic rejected

0

legitimate requests rejected

The unit of load on an inference server is the **token**, not the request. Every rate limiter in common use disagrees.

`nginx limit_req`, API-gateway requests-per-second rules and WAF throttles all count **requests**, because on a conventional web service requests cost roughly the same. On an LLM endpoint they do not: a 24-token chat reply and a 2048-token generation differ in GPU cost by nearly three orders of magnitude, and both are one well-formed request.

Who deploys this

A team self-hosting an open-weights model behind a public or partner-facing API, without per-tenant GPU isolation — the normal case, since dedicating a GPU per tenant is what self-hosting avoids.

What it replaces

It takes the position an API gateway already occupies, speaks the same OpenAI-compatible protocol, and swaps the requests-per-second rule living there for a **token-cost budget**. The endpoint URL moves by one port; the model, the server and the client are untouched.

The attack needs no volume

decode, measured on T4	0.035 s / token
one 1024-token generation	≈ 36 s
vLLM launched with	<code>--max-num-seqs 8</code>
8 requests hold the GPU for	36 s

Eight requests is a rate no plausible RPS budget would block and no anomaly rule would call a flood. No botnet, no spoofing, no volume — just a large `max_tokens`.

WHY THIS APPROACH NOW / 01

vLLM's continuous batching and paged KV cache make the scarce resource **countable** — sequence slots and KV blocks, with a known limit — rather than diffuse CPU time. There is a number to admit against.

WHY THIS APPROACH NOW / 02

vLLM exports `num_requests_running` and `num_requests_waiting` on a Prometheus endpoint, so a proxy can read the backend's **real queue** instead of inferring it from response times, which is a lagging signal.

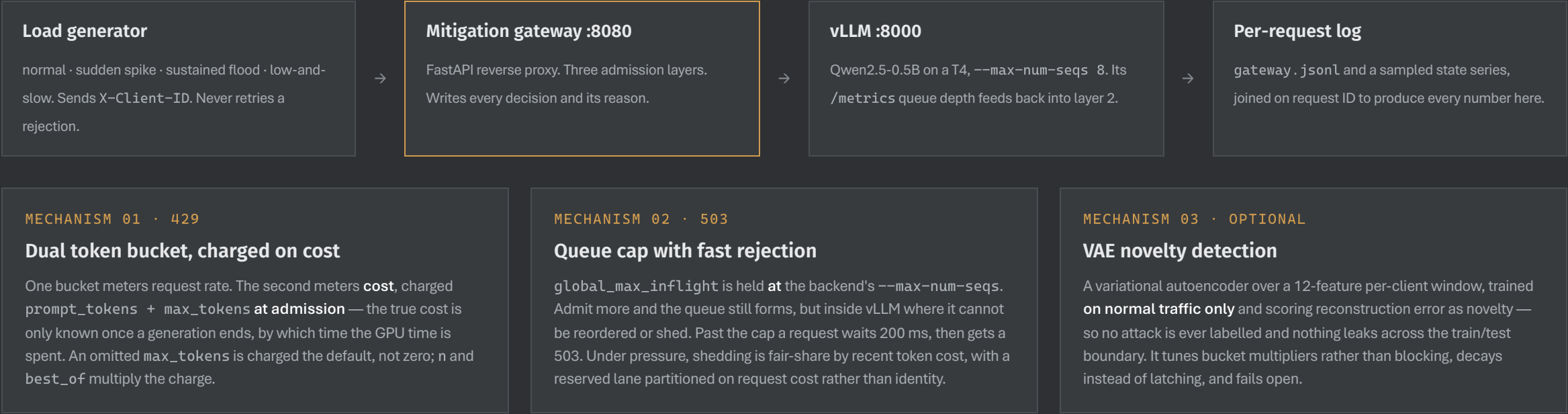
WHY NOT A LEARNED DETECTOR ALONE

Shedding has to be correct on the **first** request of an attack it has never seen, and a model cannot promise that. The two required mechanisms are deterministic and auditable; the VAE only tunes their parameters.

02

Proposed solution

two required mechanisms + one optional



03

Solution validation

four negative results, four design changes

TARGET SERVICE

Real vLLM 0.29.0 on a Tesla T4. Real weights, real Prometheus metrics.

LOAD PROFILES

Normal, sudden spike, sustained flood, low-and-slow, plus a two-client cheap-vs-expensive probe.

ABLATION

Four configs on one flag: `off`, `ratelimit`, `ratelimit_queue`, `full`.

AUTOMATED TESTS

75 unit and integration tests, including end-to-end runs over real sockets.

GPU-FREE REPRO

A stand-in calibrated to the measured T4 timing, so saturation reproduces on a laptop and in CI.

STATISTICAL CARE

Nearest-rank percentiles over successful requests only; a fast 503 is a failure, not a quick success.

The mitigation was the easy half. Most of the work went into tests that appeared to pass while measuring nothing.

- a
- The first flood did not overload anything. p50 5.603 s, p95 5.700 s, max 5.701 s — three values inside 0.1 s means one batch and no queue. Prefix caching made 50 identical prompts nearly free, and KV capacity exceeded demand 9×. Changed: randomised prompts, caching off, a concurrency sweep.
- b
- Queue depth read zero when the queue was deepest. The sampler scraped `/metrics` inside a bare `except: pass` and read a missing value as `0` — and `/metrics` is what stops answering under load. Changed: an absent snapshot is unknown, never empty, and staleness is recorded.
- c
- Identity rotation defeated every per-client defence. Eight client IDs, zero rejections, legitimate p95 4.0 s. Not a bug — per-client buckets dilute under rotation by construction. Changed: a reserved lane partitioned on request cost, which never asks who is calling.
- d
- Fair-share inverted under long requests. Cost aged out of a 10 s window while the request ran 36 s, so the attacker holding every slot was charged `0.0` and the steady cheap client measured as 100% — legitimate traffic took **45.5% rejection**. Changed: in-flight cost stays charged until completion, then is released.

REPRODUCE — NO GPU

```
pip install -e "[dev]"
pytest → 75 passed
python -m gateway.fake_vllm --max-num-seqs 8
```

REPRODUCE — ON THE GPU

Run `integration.ipynb` top to bottom: it installs vLLM, serves Qwen2.5-0.5B, then drives the same load through `configs/off.yaml` and `configs/ratelimit_queue.yaml`.

ONE RUN EXCLUDED, AND RECORDED

A pass where every request returned `ConnectError` because no gateway was listening. Its 35,128 attacker "requests" are refused connections returning instantly, not load. It sits in `excluded_runs` with the reason rather than deleted.

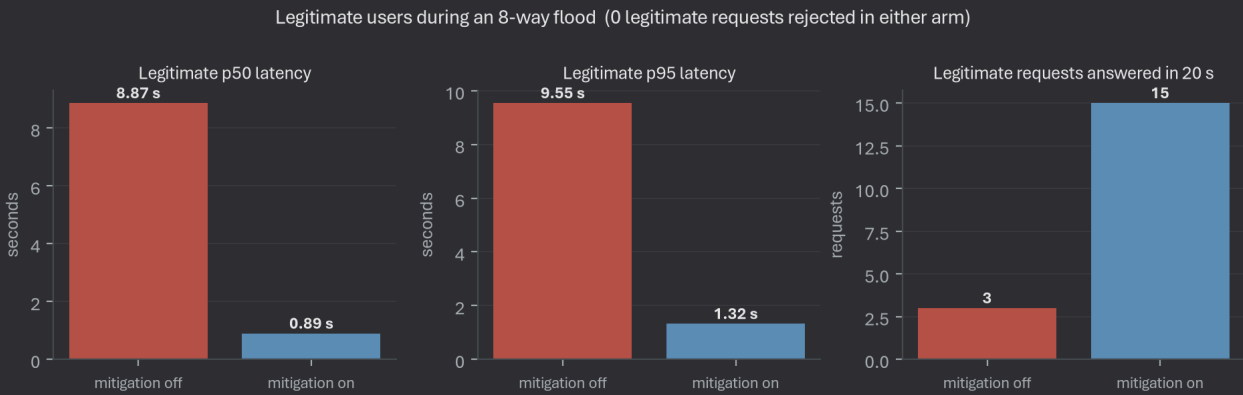
04

Results and conclusions

same load, same GPU, one config flag apart

METRIC	OFF	ON
Legitimate p95	9.546 s	1.323 s
Legitimate p50	8.867 s	0.894 s
Legitimate answered	3	15
Legitimate rejected	0	0
Attacker admitted	24	8
Attacker rejection rate	0%	50%
Attacker p95	10.355 s	12.113 s

Capacity was **reallocated**, not created. The attacker's own p95 rose while half its requests were refused: being an expensive client became slower and less reliable, which is the incentive a cost-based limiter should create.



The two left panels are the result; the right panel is what makes it credible. Latency fell **and** throughput rose **and** nothing legitimate was rejected — three facts that cannot all hold if the latency win came from shedding users. A gateway that rejects 95% of traffic has an excellent p95, so we never quote one without the other.

Rate-limit the right quantity

Eight requests, within any plausible RPS budget, took the endpoint from sub-second to nine seconds. Charging tokens at admission fixed it.

Hold the queue where you can act on it

Admitting more than `--max-num-seqs` does not remove the queue, only moves it out of reach. This mattered more than any threshold we tuned.

Partition on cost, not identity

Every identity-keyed defence dilutes under rotation. The mechanism that survived is the one that never asks who is calling.

The measurement is as easy to get wrong

Three of four design changes came from a test that proved nothing while appearing to pass. Single attacker identity on the GPU remains the open gap.